

Project Onboarding Report — Placement-Round Fraud Detection Kit

A note before anything else. This report is written against the project's last clean, committed state (git HEAD, commit af73c0c). The working directory right now is mid-merge with your teammate's branch (origin/Joshva_final-model) and has unresolved conflicts in the dashboard frontend files plus ~118 files temporarily missing from disk. None of that changes what's *supposed* to be there — this report describes the real, designed system so you have a clean map to work from once the merge is resolved. Where I couldn't verify something from code, I say so explicitly rather than guessing.

One structural fact that changes how you should read everything below: this is not one project — it's **three complete pipelines** sharing one raw dataset. Conflating them is the single easiest way to misunderstand this repo, so every section below keeps them visually separate.

1. Project Folder Structure

```
placement-round-fraud-kit/
├── data/
│   └── bank_transactions_data_2.csv    ← the ONE raw dataset every pipeline reads
├── src/
│   └── ← PIPELINE 1: supervised (v1)
│       ├── config.py                 shared paths + constants
│       ├── fe_utils.py               feature-engineering logic, training AND live-scoring
│       ├── 01_feature_engineering.py stage 1
│       ├── 02_anomaly_ensemble.py    stage 2
│       ├── 03_confidence_labeling.py stage 3
│       ├── 04_balancing.py           stage 4
│       ├── 04b_cross_validation.py   stage 4b
│       ├── 05_train_model.py         stage 5
│       └── 06_evaluation.py          stage 6
├── artifacts/
│   └── ← everything the v1 pipeline produced
├── src_research/
│   └── ← PIPELINE 2: unsupervised, in-house 46-feature set
│       ├── config_research.py, 01...13_*.py 17-phase build, one script per phase
│       ├── autoencoder_utils.py, vae_utils.py reusable PyTorch model classes
├── artifacts_research/
│   └── everything Pipeline 2 produced
├── research/
│   └── 15 human-readable phase reports (01...15_*.md)
├── src_research_v2/
│   └── ← PIPELINE 3: unsupervised, client 18-feature set
│       ├── config_research_v2.py, 04...13_*.py same 12-model methodology as Pipeline 2, different features
│       ├── autoencoder_utils.py, vae_utils.py (mirrors Pipeline 2's classes, sized for 18 features)
├── artifacts_research_v2/
│   └── everything Pipeline 3 produced
├── research_v2/
│   └── 15 phase reports, this pipeline's version
├── notebooks/
│   └── Fraud_Anomaly_Detection_Pipeline.ipynb all 17 phases as one runnable notebook (Colab-friendly)
├── dashboard/
│   └── ← THE ONLY PRODUCTION SURFACE — serves Pipeline 3 live
│       ├── backend/api_server.py      FastAPI app: the entire backend in one file
│       ├── backend/queue_state.json   persisted investigator decisions
│       ├── backend/upload_history.json persisted CSV-upload history
│       └── frontend/{index.html, css/, js/} plain HTML/CSS/JS, no framework, no build step
```

```

└─ audit/                                a full model-audit report I built earlier this session
   └─ MODEL_AUDIT_REPORT.md, generate_heatmaps.py, heatmaps/, tables/
└─ README.md, DOCUMENTATION.md, FILE_GUIDE.md, LIMITATIONS.md,
   ML_AUDIT_AFTER_FIX.md, PRESENTATION_MODEL_SUMMARY.md      ← root-level documentation

```

The files that matter most, explained

data/bank_transactions_data_2.csv — Why it exists: the single source of truth. 2,512 transactions, 495 accounts, 16 raw columns, **no fraud label anywhere**. That last fact is the reason this whole project looks the way it does — three pipelines exist because there's no ground truth to train one clean supervised model against.

src/fe_utils.py — Why it exists: this is the file that makes training and live-scoring consistent. It's split into four composable stages, each doing one job: - **add_causal_features(df)** — per-account features that only ever look at *strictly earlier* rows of the same account (safe to compute before any split exists, because moving the split boundary later can't change a value that only depends on the past). - **fit_global_stats(df)** — fits cross-transaction lookups (type averages, device/IP/merchant popularity counts) using **only the rows passed in**. Called with training rows only during model-building. - **apply_global_stats(df, stats)** — applies an already-fitted **stats** dict to *any* dataframe (val, test, or a single brand-new transaction). - **finalize_matrix(df, encoders=None)** — drops identifier columns, encodes categoricals, keeps **TransactionID** as a non-modeled join key. - **transform_new(txn, reference) / transform_batch_new(df, reference)** — the **live-scoring path**. These call the exact same **_engineer_new_row()** logic used at training time, so a transaction scored today and a transaction the model was trained on are engineered identically. This is *the* file that connects "how the model was built" to "how it's used after."

dashboard/backend/api_server.py — Why it exists: this is the entire backend. One 1,200+ line file: loads every Pipeline 3 artifact once at startup into an in-memory **STATE** dict (**_load_state()**, line 223), runs a self-check that reloaded models reproduce the published scores, then exposes ~15 REST endpoints, and finally mounts the frontend directory as static files (**app.mount("/", StaticFiles(...))**, the very last line). **Who uses it:** the browser, via **dashboard/frontend/js/api.js**. **What it connects to:** **artifacts_research_v2/*** (read-only, never retrains), plus **src/fe_utils.py** + **artifacts/xgb_model_best.json** for the one supervised-scoring code path (CSV upload in "raw format").

dashboard/frontend/js/app.js — Why it exists: all of the UI's behavior — page navigation, every API call, every table/form. No framework; DOM elements are looked up by ID and populated directly. **Who uses it:** loaded by **index.html**. **What it connects to:** **api.js** (all backend calls go through there), **charts.js** (all visualizations).

Root docs — **README.md** (quick start), **FILE_GUIDE.md** (a file-by-file index — written *before* Pipeline 3/ **research_v2/** existed, so it's incomplete now), **ML_AUDIT_AFTER_FIX.md** (a full leakage audit of Pipeline 1 — what leaked, why, and exactly how it was fixed), **LIMITATIONS.md** (the honest-caveats slide content for Pipeline 1), **DOCUMENTATION.md** (Pipeline 1's stage-by-stage technical writeup).

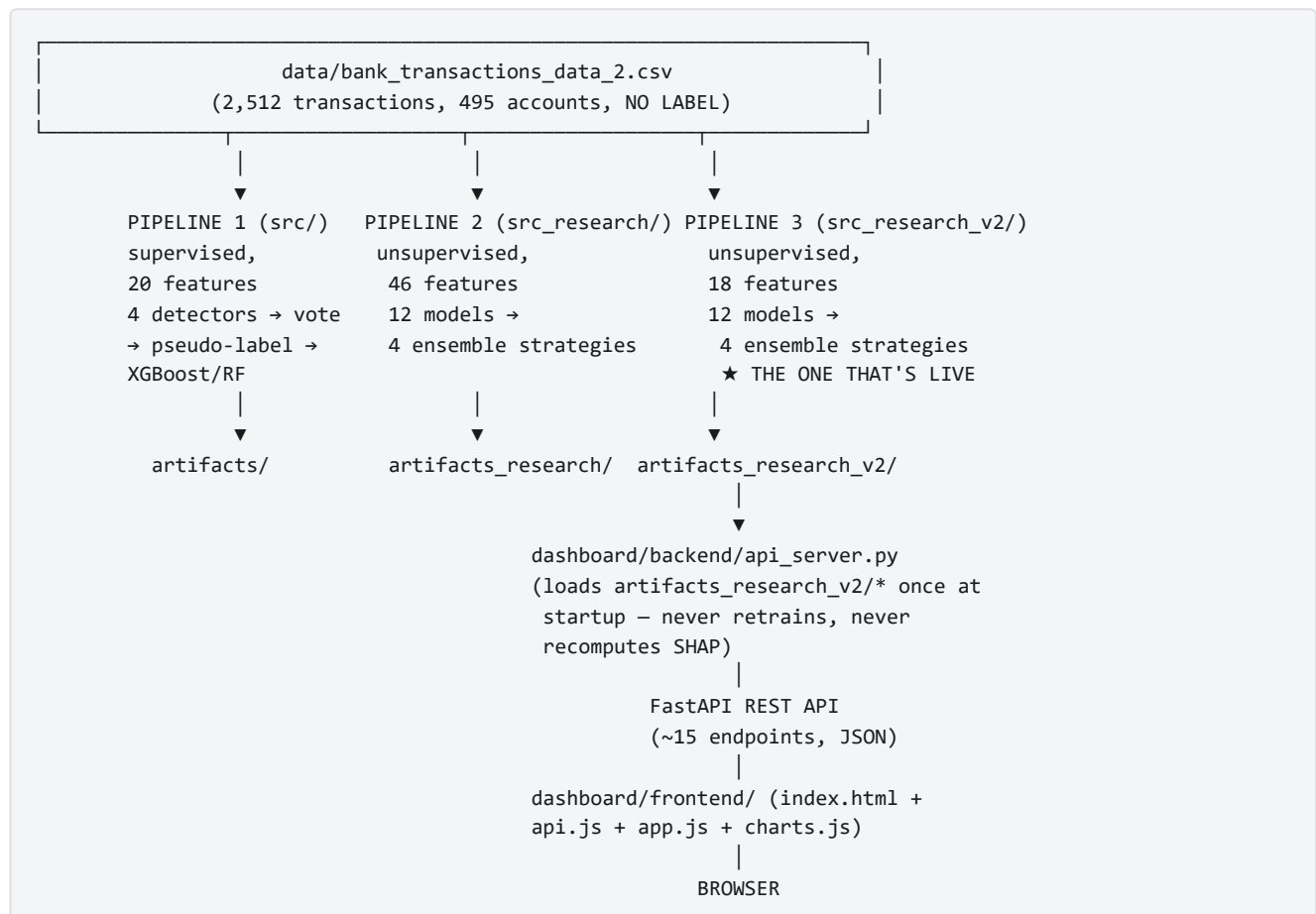
"If I open this project for the first time, these are the files I should understand first"

1. **data/bank_transactions_data_2.csv** — everything starts here; know its shape and that it has no label.
2. **ML_AUDIT_AFTER_FIX.md** — explains *why* the pipeline is architected the way it is (chronological split, train-only fitting) — read this before the code, not after.
3. **src/fe_utils.py** — the training/live-scoring contract; understand the four-stage split before reading any pipeline script.
4. **src/config.py** — every path and constant in one place; tells you what artifact each stage produces.
5. **dashboard/backend/api_server.py** — the one file that ties everything together into something runnable.
6. **research_v2/15_final_research_report.md** — the standalone executive summary of the production pipeline (Pipeline 3); reads on its own.

7. `src_research_v2/12_ensemble_scoring.py` — how 10 independent model scores become one number.
8. `dashboard/frontend/js/app.js` — how the UI turns those numbers into something a person looks at.
9. `src_research_v2/config_research_v2.py` — the exact 18-feature list and shared constants for Pipeline 3.
10. `FILE_GUIDE.md` — useful as an index, but cross-check it against the actual folder listing (it predates Pipeline 3).

2. System Architecture

Three pipelines, one dashboard. Only Pipeline 3 (research_v2) is wired into anything a user actually touches. Pipelines 1 and 2 are real, fully-executed research builds whose outputs live in the repo but aren't served live.



Frontend: plain HTML/CSS/JS. No React/Vue, no bundler, no build step — `index.html` loads `<script>` tags directly. `app.js` owns all state and DOM updates; `api.js` is a thin `fetch()` wrapper; `charts.js` draws every chart as hand-built inline SVG (no charting library).

Backend: a single FastAPI app (`api_server.py`). No database — every "table" is a CSV/JSON file read from `artifacts_research_v2/` at process startup and held in memory (`STATE`). The two exceptions that persist across restarts are `queue_state.json` (investigator Approve/Escalate/Block decisions) and `upload_history.json` (past CSV-upload runs) — both plain JSON files written to disk, not a database.

ML models: 10 active unsupervised anomaly models (research_v2), combined by percentile aggregation into one score. Isolation Forest is the only model with a computed SHAP explanation. A separate, disconnected supervised model (XGBoost, from Pipeline 1) is loaded *only* for the one CSV-upload code path that accepts raw (non-preprocessed) transaction data.

Dataset: one CSV, read three different ways by three different feature-engineering scripts, producing three different feature matrices (20 / 46 / 18 columns) — this is the single most important thing to keep straight about this repo.

API: REST, JSON, same-origin (the backend serves the frontend itself, so there's no CORS configuration and no separate API base URL).

Database: none. Confirmed by reading every import in `api_server.py` — no SQL driver, no ORM. State = files on disk, loaded into memory at boot.

Why each connection exists: - **Frontend** → **API, not Frontend** → **files directly**: the frontend never reads a CSV/pickle itself; it always goes through an endpoint, because the backend is what guarantees row-alignment across the raw data, the feature matrix, and every score file (there's a startup assertion that all of these line up by `TransactionID` — see `_load_state()`). - **API** → `artifacts_research_v2/`, **not API** → `src_research_v2/`: the backend never imports or runs the pipeline scripts at request time — it only reads their *output*. This is deliberate: scoring 2,512 transactions live on every page load would be slow and would defeat the whole point of precomputing SHAP once. The one place live computation happens is the Manual Entry / Upload path, and even there it loads a pre-trained model file (`isolation_forest.pkl`), it doesn't retrain anything. - **Pipeline 1** → **dashboard, but only for raw-CSV upload**: Pipeline 3's 18 features can't be honestly computed from a brand-new transaction with no context (they rely on population-wide frequency counts). Pipeline 1's feature set *can* be computed for one new transaction (`fe_utils.transform_new`), because its features are either raw fields or strictly-causal per-account history. So raw-format CSV upload deliberately routes to the older, smaller Pipeline 1 model instead.

3. ML / AI Pipeline — Deep Dive

3.1 Which stages actually exist

All three pipelines share this shape; only the boxes differ:

```
RAW DATA → DATA CLEANING/QUALITY → PREPROCESSING → FEATURE ENGINEERING →  
FEATURE SELECTION → MODEL TRAINING → MODEL EVALUATION → MODEL SAVING →  
MODEL LOADING → PREDICTION
```

There is no separate "feature selection" step with elimination — all engineered features are kept and fed to every model; "selection" here means *which scaler/encoding* was chosen (documented, measured comparisons — see `research_v2/05_feature_selection_and_preprocessing.md`), not dropping features.

3.2 Data

- **Dataset:** `data/bank_transactions_data_2.csv`, 2,512 rows, 16 raw columns, 0 missing values, 0 duplicates.
- **Target: none.** No fraud label exists anywhere in this dataset. This single fact is why Pipelines 2 and 3 are unsupervised, and why Pipeline 1 has to *manufacture* a pseudo-label before it can train a classifier.
- **Important raw fields:** `TransactionAmount`, `AccountBalance`, `LoginAttempts`, `TransactionDuration`, `CustomerAge`, plus identifiers (`AccountID`, `DeviceID`, `IP Address`, `MerchantID`) that get converted into counts/frequencies rather than used raw. `PreviousTransactionDate` is present but broken (an export artifact — all values cluster within minutes of one timestamp) and is dropped by every pipeline.

3.3 Preprocessing

Step	Pipeline 1	Pipelines 2 & 3
Missing values	none present	none present
Scaling	<code>StandardScaler</code> , fit on train fold only	<code>RobustScaler</code> , fit on train split only — chosen empirically because it's measurably less sensitive to the top-1% outlier rows than <code>StandardScaler</code> / <code>MinMaxScaler</code>
Encoding	one-hot (low-cardinality) + label-encode (high-cardinality, e.g. <code>Location</code>)	frequency-encoding for <code>Location</code> (measured to beat label encoding, avoids the extra columns one-hot would add)
Outlier handling	none removed — outliers are exactly what the models are looking for	same — never removed, only measured (5 different detection methods compared side by side in Pipeline 2's Phase 3)
Imbalance handling	SMOTE or class-weighting, compared head-to-head, applied to the training fold only	not applicable — no label, no class imbalance to correct

Why train-only fitting matters, everywhere: every scaler, encoder, and cross-transaction statistic (`fit_global_stats`) is fit on the training split and only *applied* to validation/test/new data. This is the fix for the leakage documented in `ML_AUDIT_AFTER_FIX.md` — fitting on the full dataset before splitting let test-set information leak backward into the training statistics.

3.4 Feature Engineering

Three different feature sets, sized 20 / 46 / 18 — this is the fact most likely to trip you up if you're not careful.

- **Pipeline 1 (20 features):** raw behavioral fields + causal per-account features (`Amount_vs_AccountAvg` , `DeviceNoveltyFlag` , `LocationNoveltyFlag` , `TimeSinceLastTxn`) + train-fit global stats (`Amount_vs_TypeAvg` , `DeviceTxnCount` , `IPTxnCount` , `MerchantTxnCount`) + categorical encodings.
- **Pipeline 2 (46 features, in-house):** Pipeline 1's 20 + 26 more — velocity counts, expanding/rolling amount baselines, amount z-scores, cyclical time encoding, cross-account device/IP sharing counts. This is the feature set with **personal-baseline statistics** — it can ask "is this unusual *for this account*."
- **Pipeline 3 (18 features, client-designated):** a *different, smaller* set — 5 raw fields, 4 **global** frequency counts (not per-account), an amount-to-balance ratio, a global high-amount flag, categorical encodings. It has **no personal-baseline features at all** — it can only ask "is this unusual in the population," never "for this customer."

Example feature explained end-to-end — `Amount_vs_AccountAvg` (Pipeline 1 & 2): computed as `(this transaction's amount - this account's prior expanding mean) / (prior mean + ε)`. The "prior expanding mean" only ever includes *strictly earlier* transactions of the *same account* (`groupby("AccountID")...shift().expanding().mean()`), which is why it's safe to compute before any split exists. It's useful because a \$500 charge means something completely different for an account that normally spends \$50 versus one that normally spends \$2,000 — this feature captures that directly, which the raw amount alone cannot.

3.5 Models

Pipeline 1 — supervised (a pseudo-label, not real fraud): 4 unsupervised detectors (Isolation Forest, LOF, One-Class SVM, Elliptic Envelope/MCD) each vote independently on the training fold; 3–4 votes = high-confidence, 2 = medium, 0–1 = normal; high+medium collapse into a binary label. That label feeds two XGBoost variants (SMOTE-balanced vs. class-weighted) and a Random Forest, all compared on validation/test. **XGBoost + Class Weighting** was selected (highest test PR-AUC). These are **sequential and independent by role** — the 4 detectors run first and only vote, they don't feed each other; their combined vote becomes the label the classifiers train against.

Pipelines 2 & 3 — 12 unsupervised anomaly models, run independently on one shared feature matrix, then combined: Isolation Forest, LOF, One-Class SVM, Elliptic Envelope, DBSCAN, HDBSCAN, K-Means, GMM, Autoencoder¹, VAE, LSTM-AE, and a rule-based Hybrid Ensemble (a vote of IF+LOF+Autoencoder). **These do not feed each other** — each independently scores every transaction on the same feature matrix; they're combined only at the very end, by 4 different mathematical strategies (weighted average, rank aggregation/Borda, percentile aggregation, PCA-stacking proxy). **Percentile aggregation is the one actually used**, selected because it's bounded, needs no tuned weights, and degrades gracefully if a model is missing.

¹ In `research_v2` /the live dashboard, the Autoencoder has been deliberately excluded from the active 10-model ensemble (least retrain-stable model measured, lowest internal-validity score of the 12) — see `src_research_v2/12_ensemble_scoring.py` 's module docstring. It still exists as a trained artifact and is shown for reference on the Model Comparison page, it's just not part of the live score.

Exactly how the final prediction is calculated (Pipeline 3, live): each of the 10 active models produces a raw anomaly score → each score is converted to a percentile rank (0 to 1) within that model's own distribution → the row-wise mean of those 10 percentiles is `ensemble_percentile_average`, a single number in (0,1) → that number is compared against two fixed cutoffs (99th percentile = Priority review, 95th percentile = Standard review, else Normal) — **no automatic blocking**, because a block decision needs a false-negative rate this project has no label to measure.

3.6 Training phase vs. prediction phase (the distinction the brief specifically asked about)

How the model was created (offline, one-time, files: `src_research_v2/04...13_*.py`):

```
raw CSV
→ 04_feature_verification.py    (verify the 18 features are correct/reproducible)
→ 05_dim_reduction.py          (PCA/UMAP/t-SNE, diagnostic only)
→ 06_models_classical.py        (fit 8 classical models on the TRAIN split)
→ 07_models_deep.py             (train Autoencoder/VAE/LSTM-AE + Hybrid Ensemble)
→ 09_hyperparameter_optimization.py (tune Isolation Forest, GMM, VAE only)
→ 10_evaluation.py              (internal validity, bootstrap stability – no label to score against)
→ 11_explainability.py          (SHAP for Isolation Forest, via TreeExplainer)
→ 12_ensemble_scoring.py        (combine 10 models → ensemble_percentile_average)
→ 13_threshold_optimization.py  (99th/95th percentile cutoffs)
→ saved to artifacts_research_v2/*.pkl, *.pt, *.csv, *.json
```

This runs once, by a person, from the command line. It produces files. It never runs when a user opens the dashboard.

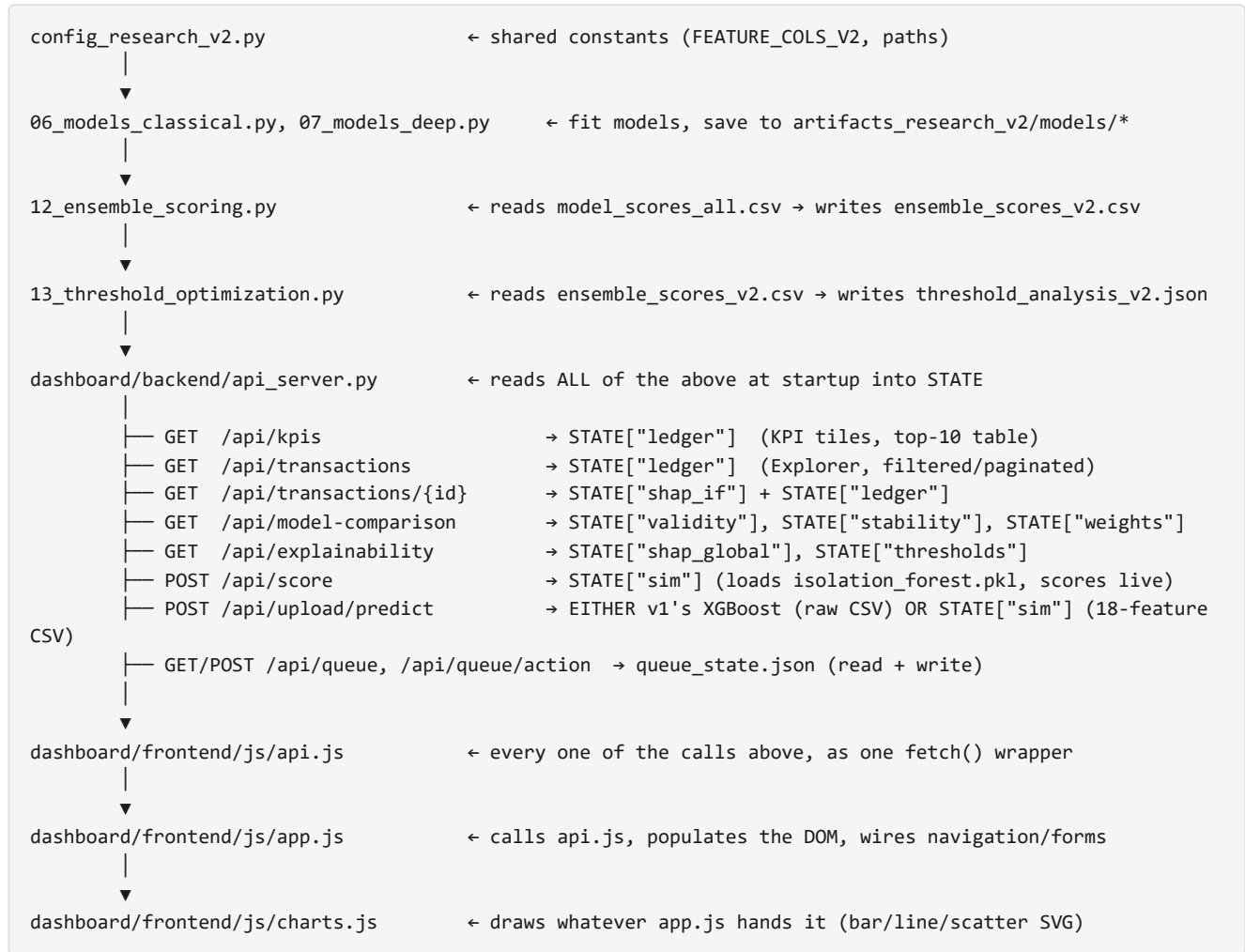
How the model is used after being created (online, every request, file: `dashboard/backend/api_server.py`):

```
server starts
→ _load_state() reads artifacts_research_v2/*.pkl/.pt/.csv/.json into memory ONCE
→ self-check: reloaded Isolation Forest reproduces the published scores exactly
→ FastAPI starts serving requests
→ a browser hits e.g. GET /api/transactions/TX000275
→ the backend looks up that row's PRECOMPUTED score/SHAP from the in-memory STATE
→ returns it as JSON – no model runs, no scoring happens on this request
```

The **one** exception where a model actually runs live is Manual Entry / raw-CSV Upload: the backend loads `isolation_forest.pkl` (already-trained, from disk) and calls `.decision_function()` on one new row's features, plus a live `shap.TreeExplainer` call — this is inference on a pre-trained model, not training.

4. Complete Connection / Execution Flow

Dependency map (Pipeline 3 + dashboard, the live path)



"Who calls this file? What does it call? What data passes between them?"

File	Called by	Calls	Data passed
config_research_v2.py	every src_research_v2/*.py script	nothing	file paths, FEATURE_COLS_V2 (a Python list of 18 strings)
06_models_classical.py	run manually, once	model_scores_all.csv write, models/*.pkl write	the 18-feature matrix in, per-model score column out
12_ensemble_scoring.py	run manually, once	reads model_scores_all.csv , model_pairwise_spearman.csv	10 score columns in, one ensemble_percentile_average column out
api_server.py _load_state()	called once at process start	reads every artifacts_research_v2/* file	nothing in, a single in-memory STATE dict out
api_server.py endpoint functions	an incoming HTTP request	read from STATE (no file I/O per- request)	a TransactionID or query params in, a JSON dict out
api.js	app.js	fetch() to the backend	JS object params in, a parsed JSON object (Promise) out
app.js	the browser (click/submit events)	api.js , charts.js , direct DOM writes	user input in, rendered HTML/SVG out

Start-to-finish walkthrough

1. **Which file starts first?** `python -m uvicorn backend.api_server:app --port 8000`, run from `dashboard/`. This imports `api_server.py`.
2. **What does it initialize?** Module-level code runs top to bottom: constants, then `STATE = _load_state()` (line 388) — this is where every artifact file gets read and the self-check runs. Then `app = FastAPI(...)` (line 405). Then every `@app.get / @app.post` decorator registers a route. Finally `app.mount("/", StaticFiles(...))` (last line) wires the frontend directory to be served at `/`.
3. **What component starts?** One process: the FastAPI/uvicorn server. There is no separate frontend build/dev-server process — the same server serves both the API and the static HTML/CSS/JS.
4. **How does the frontend start?** It doesn't "start" independently — a browser requests `http://127.0.0.1:8000/`, gets `index.html` back from `StaticFiles`, which pulls in `api.js / app.js / charts.js / icons.js / format.js` via `<script>` tags. `app.js`'s `DOMContentLoaded` handler then fires, wires every button, and calls `showPage("overview")`.
5. **How does the backend start?** Covered in step 2 — it's the same process, no separate backend startup step.
6. **When I give an input** (e.g. submit the Manual Entry form), **where does it go?** `app.js`'s form submit handler builds a JSON payload from the form fields → `Api.score(payload)` in `api.js` → `fetch("/api/score", {method:"POST", body: JSON.stringify(payload)})`.
7. **Which function receives it?** `score_scenario(body: ScenarioRequest)` in `api_server.py` (the Pydantic `ScenarioRequest` model validates the shape automatically before the function body even runs).
8. **How is it processed?** The 12 raw fields are turned into the 18 engineered feature values (same formulas as training-time feature engineering, computed inline in this function using frozen training constants), assembled into a 1×18 array, scaled with the already-fitted `RobustScaler`.
9. **Which model is called?** `sim["iforest"].decision_function(...)` — the already-trained Isolation Forest, loaded from `isolation_forest.pkl` at server startup — plus a live `shap.TreeExplainer` call for the attribution.
10. **How is the result generated?** The raw score is converted to a percentile (its rank among the 2,512 real transactions' own Isolation Forest scores), compared against the 99th/95th cutoffs for a tier, and packaged into a JSON dict with the score, tier, SHAP breakdown, and the real frequency values used.
11. **How does the result return to the user?** FastAPI serializes that dict to JSON → the browser's `fetch()` promise resolves in `api.js` → `app.js`'s `renderSimResult()` writes the values into the DOM → the user sees a badge, a number, and a chart, all on the same page, no reload.

```
START (uvicorn boots api_server.py)
→ INPUT (a browser request, or a form submission)
→ PROCESSING (feature engineering + scaling, in api_server.py or precomputed in STATE)
→ ML (a lookup into precomputed scores, OR one live Isolation Forest inference)
→ OUTPUT (JSON → fetch() → DOM update, all within the same page)
```

30-Second Explanation

"This is a fraud-detection dashboard for a bank transaction dataset that has no fraud label. Because there's nothing to train a classifier against directly, the real work is 10 different unsupervised anomaly-detection models — different mathematical ways of asking 'does this look unusual' — combined into one risk score per transaction. A FastAPI backend precomputes everything once and serves it to a plain HTML/JS dashboard where an investigator can browse, filter, and act on the highest-risk cases."

2-Minute Explanation

"There are actually three separate builds in this repo, all on the same 2,512-transaction dataset. The first is supervised: four unsupervised detectors vote on which transactions look statistically odd, that vote becomes a stand-in label, and an XGBoost model is trained against it — giving a real, measurable test-set score, but measuring how well it reproduces the detectors' own judgment, not real fraud. The other two builds are genuinely unsupervised: 12 anomaly models each, built twice on two different feature sets — one with 46 features including per-account personal-baseline statistics, one with a client-designated 18-feature set of population-level frequencies. Both compare all 12 models against each other, since there's no ground truth to evaluate against otherwise. The 18-feature version is the one that's actually deployed: its 10-model ensemble score (Isolation Forest is deliberately excluded now) feeds a FastAPI backend that precomputes every score and SHAP explanation once at startup, then serves them through a REST API to a plain-JS dashboard with an Overview page, a searchable transaction explorer, an investigation queue with persisted decisions, and a manual scenario simulator that runs one model live for a hypothetical transaction."

Technical Explanation

Three feature-engineering pipelines (20/46/18 columns) over one raw CSV, each with train-only-fit scalers/encoders/statistics to avoid leakage (verified via a documented before/after audit for the supervised build). The supervised build (Pipeline 1) trains a pseudo-label from a 4-detector consensus vote on the training fold, then compares XGBoost (SMOTE vs. class-weighted) and Random Forest on a chronological 64/16/20 train/val/test split, selecting by test PR-AUC. The two unsupervised builds (Pipelines 2/3) each fit 12 models (tree-based, density-based, distribution-based, and three neural reconstruction models) on a RobustScaler-scaled matrix, evaluate them without a label via internal validity metrics and bootstrap retrain-stability, explain two of them via SHAP, and combine model scores into a single risk number via percentile-rank averaging across models — chosen over three alternative combination strategies for its boundedness and graceful degradation on missing members. The live system (Pipeline 3, 10 active models after excluding the Autoencoder) is served by a single-process FastAPI backend that loads every artifact into memory once, self-checks reproducibility against the published scores, and answers all read requests from that in-memory state — the only live model inference happens on the Manual Entry / CSV-upload code paths, which call an already-trained Isolation Forest's `.decision_function()` plus a `TreeExplainer` SHAP call on one row at a time.

WHAT I NEED TO REMEMBER

Project

Detect anomalous/suspicious retail-banking transactions in a dataset that has **no fraud label** — so the problem is really "rank transactions by how unusual they are, and explain why," not "classify fraud."

Architecture

Three ML pipelines (`src/` , `src_research/` , `src_research_v2/`) writing to three artifact folders, one FastAPI backend (`dashboard/backend/api_server.py`) that serves only the third pipeline's precomputed outputs, one plain-JS frontend (`dashboard/frontend/`) with no build step. No database — files on disk, loaded into memory.

Dataset

`data/bank_transactions_data_2.csv` — 2,512 transactions, 495 accounts, 16 raw columns, 0 missing values, **no label**.

ML

Pipeline 1: 4 unsupervised voters (Isolation Forest, LOF, One-Class SVM, Elliptic Envelope) → pseudo-label → XGBoost/Random Forest. Pipelines 2 & 3: the same 4, plus DBSCAN, HDBSCAN, K-Means, GMM, Autoencoder, VAE, LSTM-AE, and a Hybrid Ensemble vote — 12 models total, combined by percentile aggregation (Autoencoder excluded from the live 10-model ensemble by decision).

Features

20 (Pipeline 1) / 46 (Pipeline 2, adds per-account personal-baseline stats) / 18 (Pipeline 3, adds only global population-frequency counts — no per-account memory at all). Never mix these three up.

Training

Offline, one-time, run manually stage-by-stage from the command line (`src*/0N_*.py`). Every scaler/encoder/statistic is fit on the training split only, applied unchanged to everything else.

Prediction

Two modes: (1) precomputed and served from memory for all 2,512 known transactions (no live inference); (2) live inference for a genuinely new/hypothetical transaction, using an already-trained Isolation Forest loaded from disk plus live SHAP — never retraining.

Files

1. `data/bank_transactions_data_2.csv` , 2. `ML_AUDIT_AFTER_FIX.md` , 3. `src/fe_utils.py` , 4. `dashboard/backend/api_server.py` , 5. `src_research_v2/config_research_v2.py` , 6. `src_research_v2/12_ensemble_scoring.py` , 7. `research_v2/15_final_research_report.md` , 8. `dashboard/frontend/js/app.js` .

Flow

Raw CSV → train-only-fit feature engineering → 10–12 independent anomaly models on one shared matrix → percentile-averaged into one score → thresholded into Priority/Standard/Normal → precomputed and cached → served through a REST API → rendered by plain-JS pages an investigator actually uses.

One-line architecture

Raw CSV → Feature Engineering (train-only fit) → 12 Unsupervised Models → Percentile-Aggregated Ensemble Score → FastAPI (precomputed, in-memory) → Plain-JS Dashboard → Investigator Decision